

Programming for AI

AI-2001

UML | has-a Relationship



National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2001	Introduction Programming for AI
----------------------	---------------------------------

Contents

1. UML Class Diagram?	2
1.1 UML Class Notation	2
1.1.1 Class Name:	2
1.1.2 Class Attributes:	2
1.1.3 Class Operations (Methods):	2
1.1.4 Class Visibility	3
2. has-a Relation	4
2.1 Aggregation	4
2.2 Composition	5

Talha Shahid

1. UML Class Diagram?

The UML Class diagram is a graphical notation used to construct and visualize object-oriented systems. A class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's:

1.1 UML Class Notation

A class represent a concept which encapsulates state (attributes) and behaviour (operations). Each attribute has a type. Each operation has a signature. The class name is the only mandatory information.



1.1.1 Class Name:

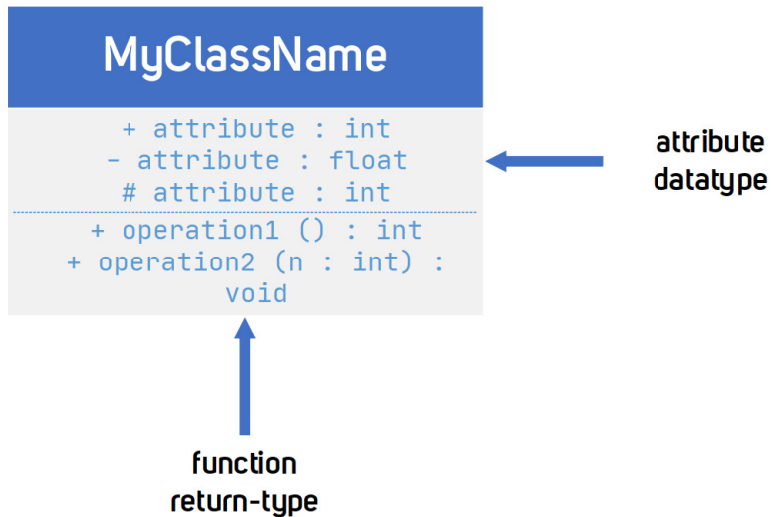
- The name of the class appears in the first partition.

1.1.2 Class Attributes:

- Attributes are shown in the second partition.
- The attribute type is shown after the colon.
- Attributes map onto member variables (data members) in code.

1.1.3 Class Operations (Methods):

- Operations are shown in the third partition. They are services the class provides.
- The return type of a method is shown after the colon at the end of the method signature.
- The return type of method parameters is shown after the colon following the parameter name. Operations map onto class methods in code.



1.1.4 Class Visibility

The +, - and # symbols before an attribute and operation name in a class denote the visibility of the attribute and operation.

- + denotes public attributes or operations
- - denotes private attributes or operations
- # denotes protected attributes or operations



2. has-a Relation

A "has-a" relationship is a specific type of association that indicates ownership or a whole-part relationship. It is often classified into aggregation and composition.

2.1 Aggregation

Aggregation is a "whole-part" relationship where the contained objects (parts) can exist independently of the container (whole). The container uses the parts but does not own them.

Example:

- A "Library" has books, but the books can exist even if the library does not.

```
class Book:
    def __init__(self, title):
        self.title = title

class Library:
    def __init__(self, library_name):
        self.library_name = library_name
        self.books = []

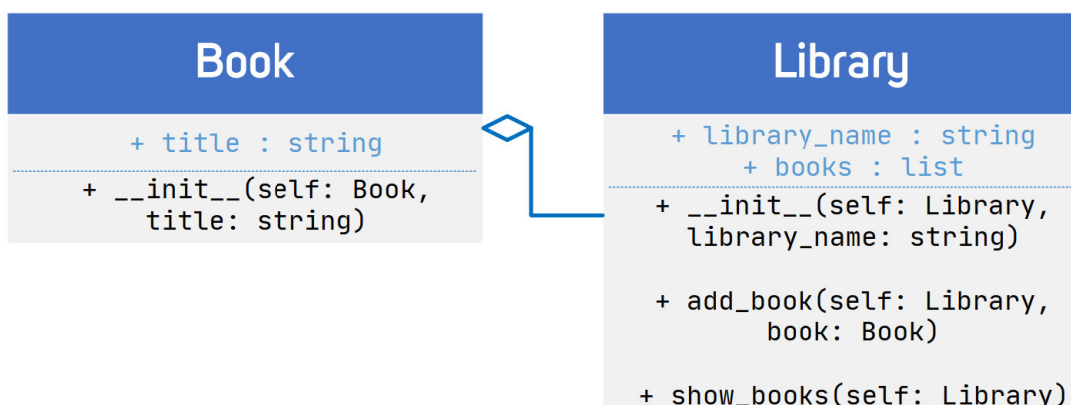
    def add_book(self, book):
        self.books.append(book)

    def show_books(self):
        book_titles = [book.title for book in self.books]
        print(f"Books in {self.library_name}: {' '.join(book_titles)}")

book1 = Book("Programing Fundamentals")
book2 = Book("Introduction to Object-Oriented Programming")

library = Library("FAST Library")
library.add_book(book1)
library.add_book(book2)
library.show_books()
```

Output	Books in FAST Library: Programing Fundamentals, Introduction to Object-Oriented Programming
---------------	---



2.2 Composition

Composition is a stronger "whole-part" relationship where the part's lifecycle is tied to the whole. If the container (whole) is destroyed, the contained objects (parts) are also destroyed.

Example:

A "Car" has an "Engine." If the car is destroyed, the engine is also discarded.

```
class Engine:
    def __init__(self, horsepower, fuel_type):
        """Initialize the engine with horsepower and fuel type."""
        self.horsepower = horsepower    # The power of the engine
        self.fuel_type = fuel_type      # The type of fuel the engine uses

    def start(self):
        """Simulate starting the engine."""
        print(f"Engine with {self.horsepower} HP running on {self.fuel_type} started.")

    def stop(self):
        """Simulate stopping the engine."""
        print(f"Engine with {self.horsepower} HP stopped.")

class Car:
    def __init__(self, make, model):
        """Initialize the car with make, model, and create an empty dictionary for engines."""
        self.make = make    # The make of the car (e.g., Toyota)
        self.model = model  # The model of the car (e.g., Corolla)
        # Dictionary to store engines by type
        self.engines = {}

    def add_engine(self, engine_type, horsepower, fuel_type):
        """Add an engine to the car using a dictionary."""
        self.engines[engine_type] = Engine(horsepower, fuel_type)

    def start_engine(self, engine_type):
        """Start a specific engine based on its type."""
        if engine_type in self.engines:
            print(f"Starting the {self.make} {self.model} with {engine_type} engine...")
            self.engines[engine_type].start()
        else:
            print(f"Engine type '{engine_type}' not found.")

    def stop_engine(self, engine_type):
        """Stop a specific engine based on its type."""
        if engine_type in self.engines:
            print(f"Stopping the {self.make} {self.model} with {engine_type} engine...")
            self.engines[engine_type].stop()
```

```

        else:
            print(f"Engine type '{engine_type}' not found.")

    def display_info(self):
        """Display information about the car and its engines."""
        print(f"Car Make: {self.make}")
        print(f"Car Model: {self.model}")
        print("Engines available:")
        for engine_type, engine in self.engines.items():
            print(f"    - {engine_type}: {engine.horsepower} HP, Fuel Type: {engine.fuel_type}")

# Example usage
my_car = Car("Toyota", "Corolla")

# Adding engines to the car
my_car.add_engine("Standard", 132, "Gasoline")
my_car.add_engine("Sport", 150, "Premium Gasoline")

# Display car and engine details
my_car.display_info()

# Start and stop engines
my_car.start_engine("Standard")
my_car.stop_engine("Standard")

my_car.start_engine("Sport")
my_car.stop_engine("Sport")

# Attempting to start a non-existent engine
my_car.start_engine("Electric") # This should show an error message

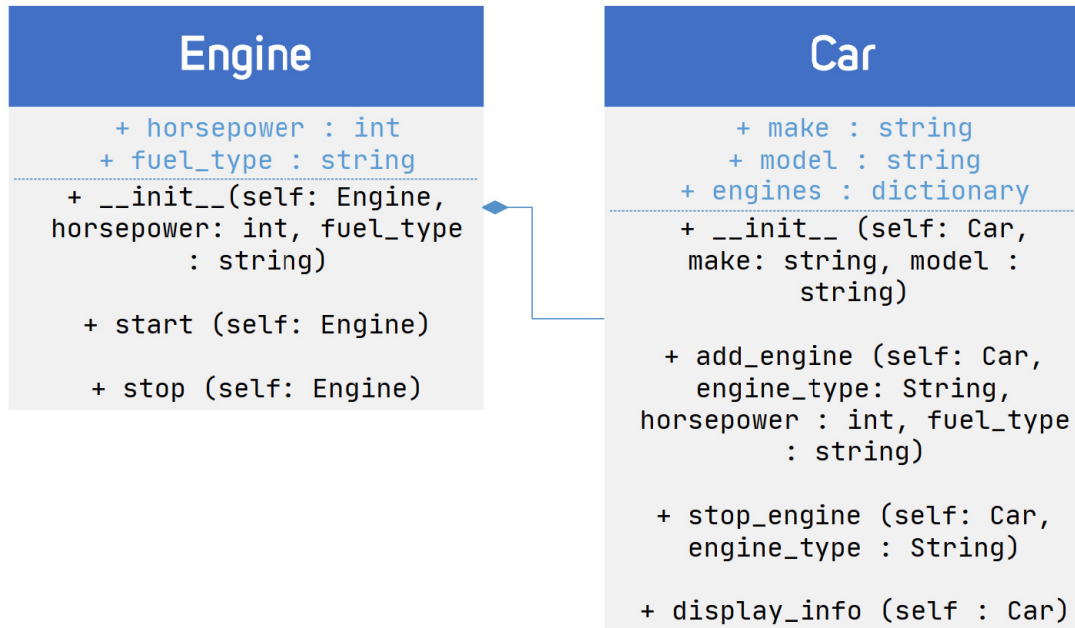
```

Output

```

Car Make: Toyota
Car Model: Corolla
Engines available:
  - Standard: 132 HP, Fuel Type: Gasoline
  - Sport: 150 HP, Fuel Type: Premium Gasoline
Starting the Toyota Corolla with Standard engine...
Engine with 132 HP running on Gasoline started.
Stopping the Toyota Corolla with Standard engine...
Engine with 132 HP stopped.
Starting the Toyota Corolla with Sport engine...
Engine with 150 HP running on Premium Gasoline started.
Stopping the Toyota Corolla with Sport engine...
Engine with 150 HP stopped.
Engine type 'Electric' not found.

```



Talha Shahid